Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра проектирования информационно-компьютерных систем
Дисциплина Технологии проектирования сложных информационных систем

Отчет
по лабораторной работе
№5 на тему:

ПРОЕКТИРОВАНИЕ АРХИТЕКТУРНЫХ РЕШЕНИЙ

Выполнил _____ Лихтар И.В.
(подпись) гр. 314301

Проверил _____ Ефремова А.Ю.
(подпись)

Минск, 2026

1 АРХИТЕКТУРА ПРОГРАММНОГО СРЕДСТВА

В качестве основного инструмента формализации архитектурных решений была выбрана нотация **C4 Model**. Данная нотация позволяет представить программное средство на нескольких уровнях абстракции: от общего системного контекста до уровня отдельных компонентов и классов. В рамках лабораторной работы архитектура программного средства WorkPulse будет представлена на четырех уровнях: контекстном, контейнерном, компонентном и кодовом. Использование C4 Model соответствует требованиям лабораторной работы №5, где необходимо описать архитектурные решения и выбрать стек технологий для реализации проекта.

Программное средство **WorkPulse** представляет собой корпоративное веб-приложение, предназначенное для автоматизации учета рабочего времени, анализа активности сотрудников, выявления простоев и формирования управленческой аналитики. Основными пользователями системы являются руководители подразделений, HR-специалисты, топ-менеджмент, специалисты по информационной безопасности и системные администраторы.

1.1 Контекстный уровень представления архитектуры

На первом этапе проектирования была разработана **контекстная диаграмма WorkPulse**. Контекстный уровень показывает программное средство как единую систему и отражает его взаимодействие с внешними пользователями и внешними информационными системами.

Данный уровень позволяет определить границы ответственности WorkPulse и показать, какие участники взаимодействуют с системой. На контекстной диаграмме не раскрывается внутренняя структура программного средства, так как основная цель данного уровня — показать место WorkPulse в корпоративной среде организации.

В рамках контекстного уровня выделены следующие внешние участники:

Сотрудник — пользователь, чья рабочая активность фиксируется системой. Сотрудник начинает рабочую сессию, использует рабочую станцию, уходит на обеденный перерыв и завершает рабочий день.

HR-специалист — пользователь, который анализирует рабочее время сотрудников, просматривает нарушения, формирует отчеты и участвует в настройке политик мониторинга.

Руководитель подразделения — пользователь, который контролирует эффективность сотрудников своего подразделения, анализирует KPI, простои и нарушения рабочего режима.

Топ-менеджмент — пользователь, который получает сводные аналитические показатели по подразделениям и использует их для принятия управленческих решений.

Специалист по ИБ / compliance — пользователь, который контролирует соблюдение требований конфиденциальности, прозрачности мониторинга и защиты данных.

Системный администратор — пользователь, который управляет учетными записями, ролями, правами доступа и техническими параметрами системы.

HRM / ERP-система — внешняя корпоративная система, из которой WorkPulse получает сведения о сотрудниках, подразделениях, должностях и организационной структуре.

На рисунке 1 представлена контекстная диаграмма программного средства WorkPulse в нотации C4.

Рисунок 1 – Контекстная диаграмма WorkPulse в нотации C4

Контекстная диаграмма показывает, что программное средство WorkPulse является центральной системой для учета рабочего времени и анализа эффективности сотрудников. Сотрудник выступает источником данных об активности, HR-специалист и руководитель подразделения используют систему для анализа дисциплины и производительности, топ-менеджмент получает сводную управленческую аналитику, а специалист по ИБ и системный администратор обеспечивают безопасную и корректную эксплуатацию системы. Внешняя HRM / ERP-система используется как источник актуальных сведений о сотрудниках, подразделениях и должностях.

1.2 Контейнерный уровень представления архитектуры

Для более детального представления внутренней организации программного средства WorkPulse был выполнен переход к **контейнерному уровню декомпозиции**. Данный уровень показывает, из каких крупных технических частей состоит система, какие задачи выполняет каждый контейнер и каким образом контейнеры взаимодействуют между собой.

В архитектуре WorkPulse выделены следующие основные контейнеры:

1. **Web Application** — веб-приложение для HR-специалистов, руководителей подразделений, топ-менеджмента, специалистов по ИБ и системных администраторов.
2. **Client Agent** — клиентский агент, устанавливаемый на рабочие станции сотрудников для сбора данных об активности.
3. **API Application** — серверное приложение, реализующее основную бизнес-логику системы.
4. **Analytics Service** — сервис аналитической обработки данных, классификации активности, выявления простоев и расчета KPI.
5. **Notification Service** — сервис отправки уведомлений о нарушениях, ошибках синхронизации и важных событиях.
6. **Integration Adapter** — модуль интеграции с внешними HRM- и ERP-системами.
7. **Database** — централизованная база данных для хранения сведений о

пользователях, рабочих сессиях, активности, нарушениях, политиках мониторинга и отчетах.

Контейнерный уровень позволяет показать техническую структуру WorkPulse без перехода к деталям реализации отдельных классов и методов. На данном уровне также фиксируются основные протоколы взаимодействия между частями системы: HTTPS/JSON для обмена между веб-интерфейсом, клиентским агентом и API, а также SQL/TCP для взаимодействия серверной части с базой данных.

На рисунке 2 представлена контейнерная диаграмма программного средства WorkPulse в нотации C4.

Рисунок 2 – Контейнерная диаграмма WorkPulse в нотации C4

Контейнерная диаграмма показывает, что WorkPulse реализуется как распределенная клиент-серверная система. Пользователи взаимодействуют с системой через веб-приложение, а сбор активности сотрудников выполняется отдельным клиентским агентом, установленным на рабочей станции. Центральным элементом архитектуры является API Application, которое принимает запросы от веб-интерфейса и клиентского агента, управляет бизнес-логикой и обеспечивает доступ к данным.

Аналитическая обработка вынесена в отдельный Analytics Service. Это позволяет изолировать ресурсоемкие операции классификации активности, выявления простоев, расчета KPI и формирования нарушений от основной серверной логики. Для хранения данных используется PostgreSQL, обеспечивающая надежное централизованное хранение сведений о пользователях, рабочих сессиях, активности, нарушениях и отчетах.

Интеграционный модуль обеспечивает получение актуальных сведений о сотрудниках, подразделениях и должностях из внешних HRM- и ERP-систем. Notification Service используется для информирования пользователей о выявленных нарушениях, ошибках синхронизации и других важных событиях. Такое разделение системы на контейнеры повышает масштабируемость, упрощает сопровождение и позволяет развивать отдельные части WorkPulse независимо друг от друга.

1.3 Компонентный уровень представления архитектуры

Для детализации внутренней логики аналитической обработки был выполнен переход к **компонентному уровню декомпозиции**. На данном уровне контейнер **Analytics Service** раскрывается как набор функциональных компонентов, каждый из которых отвечает за отдельную часть обработки данных о рабочем времени и активности сотрудников.

Выбор Analytics Service для детализации обусловлен тем, что данный контейнер реализует ключевую бизнес-логику WorkPulse: классификацию

активности, определение простоев, выявление нарушений рабочего режима и расчет KPI-показателей. Именно эти функции обеспечивают переход от простого сбора данных к формированию управленческой аналитики.

В составе **Analytics Service** выделены следующие компоненты:

1. **Activity Event Receiver** — компонент приема событий активности от серверного приложения.
2. **Work Session Processor** — компонент обработки рабочих сессий и обеденных перерывов.
3. **Activity Classification Service** — компонент классификации активности сотрудника.
4. **Idle Detection Service** — компонент выявления периодов простоя.
5. **Violation Detection Service** — компонент определения нарушений рабочего режима.
6. **KPI Calculation Service** — компонент расчета показателей эффективности.
7. **Report Data Provider** — компонент подготовки данных для отчетов и KPI-дашбордов.
8. **Monitoring Policy Provider** — компонент получения правил мониторинга.
9. **Analytics Repository** — компонент взаимодействия с базой данных.

На рисунке 3 представлена компонентная диаграмма контейнера **Analytics Service** в нотации C4.

Рисунок 3 – Компонентная диаграмма Analytics Service в нотации C4

Компонентная диаграмма показывает внутреннюю структуру контейнера Analytics Service. Входной точкой аналитической обработки является компонент Activity Event Receiver, который получает события активности и данные о рабочих сессиях от API Application. Далее данные передаются в компоненты обработки рабочих сессий и классификации активности.

Компонент Activity Classification Service определяет тип активности сотрудника на основе правил мониторинга. Idle Detection Service выявляет периоды отсутствия активности, а Violation Detection Service анализирует простои, нецелевую активность и отклонения от рабочего графика. На основе обработанных данных KPI Calculation Service рассчитывает показатели эффективности рабочего времени, которые затем используются Report Data Provider для подготовки отчетов и дашбордов.

Компонент Monitoring Policy Provider обеспечивает доступ к правилам мониторинга, лимитам простоев, рабочим графикам и справочникам категорий активности. Analytics Repository выполняет роль слоя доступа к данным и обеспечивает сохранение результатов анализа, нарушений и KPI-показателей в базе данных. Такая компонентная структура позволяет изолировать отдельные части аналитической логики и упростить дальнейшее сопровождение WorkPulse.

1.4 Уровень кода

Финальным уровнем проектирования в C4 Model является **уровень кода**. Данный уровень позволяет детализировать реализацию наиболее важного функционального фрагмента системы на уровне классов, их атрибутов, методов и связей.

В рамках программного средства WorkPulse для детализации был выбран механизм выявления простоев и нарушений рабочего режима. Данный фрагмент является одним из ключевых, поскольку именно он преобразует собранные данные об активности сотрудника в управленчески значимую информацию: простои, нарушения графика, превышение обеденного перерыва, длительную нецелевую активность и KPI-показатели.

На уровне кода выделены следующие основные классы:

1. **ViolationDetectionService** — сервис выявления нарушений рабочего режима.
2. **IdleDetectionService** — сервис определения периодов простоя.
3. **ActivityClassificationService** — сервис классификации активности сотрудника.
4. **MonitoringPolicy** — сущность, содержащая правила мониторинга.
5. **WorkSession** — сущность рабочей сессии сотрудника.
6. **ActivityEvent** — сущность события активности сотрудника.
7. **ViolationRecord** — сущность зафиксированного нарушения.
8. **KpiMetric** — сущность рассчитанного KPI-показателя.
9. **ViolationRepository** — компонент доступа к данным нарушений.

На рисунке 4 представлен уровень кода для логики выявления простоев и нарушений рабочего режима.

Рисунок 4 – Уровень кода логики выявления простоев и нарушений WorkPulse

Диаграмма уровня кода показывает структуру классов, участвующих в реализации логики выявления простоев и нарушений рабочего режима. Центральным элементом является класс **ViolationDetectionService**, который анализирует рабочую сессию сотрудника, использует правила мониторинга и формирует записи о нарушениях.

Класс **IdleDetectionService** отвечает за определение периодов отсутствия активности на основе событий **ActivityEvent**. Класс **ActivityClassificationService** классифицирует действия сотрудника и определяет, относится ли активность к продуктивной, нецелевой или нейтральной категории. Сущность **MonitoringPolicy** содержит параметры, по которым выполняется анализ: рабочий график, допустимая длительность простоя, длительность обеденного перерыва и списки приложений.

Сущность **WorkSession** хранит данные о рабочем дне сотрудника и содержит набор событий активности. При выявлении отклонения от правил

система создает объект **ViolationRecord**, который сохраняется через **ViolationRepository**. В дальнейшем данные о нарушениях используются для расчета показателей **KpiMetric**, отражающих эффективность использования рабочего времени сотрудника или подразделения.

Таким образом, уровень кода демонстрирует внутреннюю реализацию ключевого алгоритма WorkPulse. Представленная структура позволяет отделить анализ активности, выявление простоев, применение политик мониторинга и сохранение нарушений, что повышает сопровождаемость и расширяемость программного средства.

2 ТАБЛИЦЫ РЕЗУЛЬТАТОВ АНАЛИЗА ТЕХНОЛОГИЙ ДЛЯ РАЗРАБОТКИ ПРОГРАММНОГО СРЕДСТВА

2.1 Анализ библиотек и фреймворков для разработки пользовательского интерфейса

Выбор технологии для разработки пользовательского интерфейса является важным этапом проектирования программного средства WorkPulse. Интерфейс системы должен обеспечивать удобную работу HR-специалистов, руководителей подразделений, топ-менеджмента, специалистов по информационной безопасности и системных администраторов.

Для WorkPulse пользовательский интерфейс должен поддерживать отображение аналитических отчетов, KPI-дашбордов, журнала нарушений, карточек сотрудников и раздела настройки политик мониторинга. Поэтому frontend-технология должна обеспечивать высокую скорость отклика, удобную работу с таблицами и графиками, поддержку компонентного подхода и возможность строгой типизации.

В качестве потенциальных решений были рассмотрены наиболее распространенные технологии: **React**, **Angular** и **Vue.js**. Результаты сравнения представлены в таблице 1.

Таблица 1 – Сравнительный анализ Frontend-технологий

Характеристика	React	Angular	Vue.js
Производительность	Высокая за счет Virtual DOM и гибкой оптимизации компонентов	Высокая, но фреймворк более тяжеловесный	Высокая за счет легковесности
Поддержка TypeScript	Отличная, широко используется в корпоративной разработке	Нативная, TypeScript используется по умолчанию	Хорошая
Удобство создания дашбордов	Высокое, доступно большое количество библиотек для графиков и таблиц	Высокое, но требует более сложной настройки	Среднее, экосистема меньше
Кривая обучения	Средняя	Высокая	Низкая
Гибкость архитектуры	Высокая, можно самостоятельно выбирать структуру проекта	Средняя, структура проекта более жесткая	Высокая
Экосистема	Очень большая, много UI-библиотек и инструментов	Большая, но более формализованная	Средняя, активно развивается
Подход к разработке	Компонентный подход, удобный для построения интерфейсов отчетности	Полноценный фреймворк для крупных enterprise-приложений	Легковесный фреймворк для быстрой разработки

На основании данных таблицы 1 для реализации пользовательского интерфейса WorkPulse выбран **React в сочетании с TypeScript**. Данный выбор обусловлен высокой производительностью, развитой экосистемой, удобством построения аналитических дашбордов и возможностью создания переиспользуемых компонентов интерфейса. TypeScript снижает вероятность ошибок при передаче и обработке данных отчетов, KPI-показателей, пользовательских ролей и настроек мониторинга.

2.2 Анализ серверных платформ для реализации бизнес-логики

Серверная часть WorkPulse должна обеспечивать обработку данных активности, управление рабочими сессиями, применение политик мониторинга, выявление нарушений, расчет KPI, разграничение прав доступа и интеграцию с внешними HRM- и ERP-системами.

Для реализации серверной части были рассмотрены три технологические платформы: **ASP.NET Core**, **Java Spring Boot** и **Node.js / NestJS**. Сравнение представлено в таблице 2.

Таблица 2 – Сравнительный анализ Backend-платформ

Характеристика	ASP.NET Core	Java Spring Boot	Node.js / NestJS
Производительность	Очень высокая, подходит для корпоративных веб-сервисов	Высокая, подходит для enterprise-систем	Высокая при работе с большим числом I/O-запросов
Типизация	Строгая типизация C#	Строгая типизация Java	TypeScript обеспечивает типизацию, но среда выполнения остается JavaScript
Безопасность	Высокая, есть встроенные механизмы авторизации, аутентификации и политики доступа	Высокая, развитая экосистема Spring Security	Средняя/высокая, зависит от качества настройки
Работа с БД	Entity Framework Core, Dapper	Hibernate / JPA	TypeORM, Prisma, Sequelize
Фоновые сервисы	Удобная поддержка Worker Service и Background Tasks	Поддержка scheduled/background jobs	Поддержка через очереди и отдельные worker-процессы
Интеграция с корпоративными системами	Высокая, хорошо подходит для REST API и enterprise-интеграций	Очень высокая, enterprise-стандарт	Высокая, удобно строить API-шлюзы
Масштабируемость	Высокая	Очень высокая	Высокая
Сложность разработки	Средняя	Средняя/высокая	Средняя

На основании данных таблицы 2 для реализации серверной части WorkPulse выбрана платформа **ASP.NET Core**. Данный выбор обусловлен высокой производительностью, строгой типизацией языка C#, удобными средствами построения REST API, поддержкой фоновых сервисов и развитым механизмом ролевой авторизации. Для WorkPulse это особенно важно, так как система обрабатывает корпоративные данные, требует разграничения доступа между ролями пользователей и выполняет регулярную аналитическую обработку данных активности.

2.3 Анализ систем управления базами данных

Программное средство WorkPulse должно хранить структурированные данные о пользователях, ролях, рабочих сессиях, событиях активности, политиках мониторинга, нарушениях, отчетах и KPI-показателях. Для такой системы важны надежность, транзакционная целостность, поддержка сложных запросов и возможность эффективной аналитической выборки данных.

В качестве потенциальных СУБД были рассмотрены **PostgreSQL**, **MySQL** и **MongoDB**. Результаты сравнения представлены в таблице 3.

Таблица 3 – Сравнительный анализ систем управления базами данных

Характеристика	PostgreSQL	MySQL	MongoDB
Тип данных	Реляционная объектно-реляционная СУБД	Реляционная СУБД	Документоориентированная СУБД
Целостность данных	Очень высокая, поддержка ACID	Высокая, поддержка ACID	Средняя/высокая, зависит от модели данных
Поддержка сложных связей	Высокая, удобна для связей между сотрудниками, сессиями, нарушениями и отчетами	Средняя/высокая	Низкая для сложных реляционных связей
Аналитические запросы	Высокая эффективность при правильной индексации	Средняя/высокая	Подходит для гибких документов, но сложнее для реляционной аналитики
Масштабируемость	Высокая	Высокая	Высокая
Поддержка JSON	Есть JSON/JSONB	Есть JSON	Нативная
Надежность транзакций	Очень высокая	Высокая	Средняя/высокая
Удобство для WorkPulse	Оптимально подходит для структурированных корпоративных данных и отчетности	Подходит, но менее гибка для сложной аналитики	Подходит для гибких событий, но хуже для строгой структуры отчетности

На основании данных таблицы 3 для хранения данных WorkPulse выбрана **PostgreSQL**. Данная СУБД подходит для хранения взаимосвязанных корпоративных данных, обеспечивает транзакционную целостность и поддерживает сложные SQL-запросы, необходимые для формирования отчетов, KPI-дашбордов и выборок по сотрудникам, подразделениям и периодам. Дополнительным преимуществом PostgreSQL является поддержка JSONB, что позволяет хранить расширенные параметры событий активности и настроек мониторинга без постоянного изменения структуры таблиц.

3 ОПИСАНИЕ ВЫБРАННЫХ ТЕХНОЛОГИЙ ДЛЯ РАЗРАБОТКИ С ОБОСНОВАНИЕМ ИХ ВЫБОРА

На основании проведенного сравнительного анализа технологий и специфики задач, стоящих перед программным средством WorkPulse, был сформирован технологический стек, обеспечивающий надежную работу системы учета рабочего времени, анализа активности сотрудников, выявления простоев и формирования управленческой аналитики.

В качестве основного инструмента разработки пользовательского интерфейса выбрана библиотека **React** в сочетании с языком **TypeScript**. Данный выбор обусловлен тем, что WorkPulse должен предоставлять пользователям удобный веб-интерфейс для просмотра отчетов, KPI-дашбордов, журнала нарушений, карточек сотрудников и настройки политик мониторинга. React позволяет строить интерфейс на основе переиспользуемых компонентов, что упрощает разработку и дальнейшее сопровождение системы. Использование TypeScript обеспечивает строгую типизацию данных, что особенно важно при работе с отчетами, ролями пользователей, фильтрами, настройками мониторинга и аналитическими показателями.

Для реализации серверной части выбрана платформа **ASP.NET Core**. Она подходит для разработки корпоративных веб-сервисов, так как обеспечивает высокую производительность, надежную обработку HTTP-запросов, удобную реализацию REST API и поддержку механизмов аутентификации и авторизации. Для WorkPulse это является важным преимуществом, поскольку система должна разграничивать доступ между разными категориями пользователей: сотрудниками, HR-специалистами, руководителями подразделений, топ-менеджментом, специалистами по ИБ и системными администраторами.

Дополнительно ASP.NET Core позволяет эффективно реализовать фоновые процессы, необходимые для аналитической обработки данных. В рамках WorkPulse такие процессы могут использоваться для классификации активности, выявления простоев, фиксации нарушений рабочего режима, расчета KPI и подготовки данных для отчетов. Это позволяет вынести ресурсоемкую обработку из основного пользовательского сценария и повысить стабильность работы системы.

Для клиентского агента сбора активности выбрана технология **.NET Worker Service**. Данный компонент устанавливается на рабочую станцию сотрудника и отвечает за фиксацию событий активности, периодов отсутствия активности, начала и окончания рабочей сессии. Использование .NET Worker Service позволяет реализовать фоновое приложение, которое работает независимо от веб-интерфейса и передает данные на серверную часть WorkPulse по защищенному каналу связи.

В качестве системы управления базами данных выбрана **PostgreSQL**. Данная СУБД подходит для хранения структурированных корпоративных данных: пользователей, ролей, подразделений, рабочих сессий, событий активности, политик мониторинга, нарушений, отчетов и KPI-показателей. PostgreSQL обеспечивает транзакционную целостность, надежность хранения и

поддержку сложных SQL-запросов, необходимых для аналитических выборок по сотрудникам, подразделениям и периодам. Дополнительным преимуществом является поддержка JSONB, что позволяет хранить расширенные параметры событий активности и настроек мониторинга.

Межмодульное взаимодействие между веб-приложением, клиентским агентом, серверной частью, аналитическим сервисом и интеграционным модулем будет реализовано с использованием архитектурного стиля **REST** и формата обмена данными **JSON**. Такой подход обеспечивает простоту интеграции, независимость клиентской и серверной частей, а также удобство расширения системы при добавлении новых модулей.

Для обеспечения безопасности в WorkPulse будет использоваться механизм **JWT-аутентификации** и **ролевого разграничения доступа**. Это позволит ограничить доступ пользователей только к тем функциям и данным, которые соответствуют их роли. Например, сотрудник не должен иметь доступ к сводной аналитике по подразделению, руководитель должен видеть данные только своих сотрудников, а специалист по ИБ должен иметь доступ к настройкам конфиденциальности и журналу событий.

Для интеграции с внешними корпоративными системами используется отдельный **Integration Adapter**, который обеспечивает обмен данными с HRM- и ERP-системами. Через данный модуль WorkPulse может получать сведения о сотрудниках, подразделениях, должностях и организационной структуре. Это позволяет использовать актуальные кадровые данные при формировании отчетов, настройке прав доступа и анализе эффективности подразделений.

Таким образом, выбранный технологический стек включает **React + TypeScript** для пользовательского интерфейса, **ASP.NET Core** для серверной бизнес-логики, **.NET Worker Service** для клиентского агента, **PostgreSQL** для хранения данных, а также **REST/JSON** и **JWT/RBAC** для взаимодействия и безопасности. Такая комбинация технологий обеспечивает надежность, масштабируемость, безопасность и удобство сопровождения программного средства WorkPulse.

ВЫВОД

В ходе выполнения лабораторной работы было проведено проектирование архитектурных решений для программного средства WorkPulse, предназначенного для автоматизации учета рабочего времени, анализа активности сотрудников, выявления простоев и формирования управленческой аналитики. Архитектура системы была представлена с использованием нотации C4 Model, что позволило последовательно рассмотреть программное средство на разных уровнях детализации: от общего взаимодействия с внешними пользователями до структуры отдельных классов.

На первом уровне была разработана контекстная диаграмма, отражающая взаимодействие WorkPulse с основными пользователями системы: сотрудником, HR-специалистом, руководителем подразделения, топ-менеджментом, специалистом по ИБ, системным администратором, а также с внешней HRM/ERP-системой. Данный уровень позволил определить границы программного средства и его место в корпоративной информационной среде.

На втором уровне была построена контейнерная диаграмма, в которой WorkPulse представлен как распределенная клиент-серверная система. В составе архитектуры были выделены веб-приложение, клиентский агент сбора активности, серверное API-приложение, аналитический сервис, сервис уведомлений, интеграционный адаптер и база данных. Такое разделение позволяет изолировать основные части системы и обеспечить удобство дальнейшего сопровождения.

На третьем уровне была разработана компонентная диаграмма аналитического сервиса. Были выделены компоненты приема событий активности, обработки рабочих сессий, классификации активности, выявления простоев, определения нарушений, расчета KPI, подготовки данных для отчетов и взаимодействия с базой данных. Данный уровень показал внутреннюю организацию ключевой бизнес-логики WorkPulse.

На четвертом уровне был представлен уровень кода, раскрывающий структуру классов, участвующих в реализации логики выявления простоев и нарушений рабочего режима. Центральным элементом выступает сервис выявления нарушений, который использует данные рабочей сессии, события активности, правила мониторинга и сохраняет сформированные записи о нарушениях для последующего анализа и расчета KPI.

Таким образом, в лабораторной работе была сформирована полноценная архитектурная основа программного средства WorkPulse. Разработанные архитектурные решения позволяют перейти к последующим этапам проектирования пользовательского интерфейса, структуры базы данных и реализации программных модулей. Выбранный подход обеспечивает надежную работу системы, удобство сопровождения и возможность дальнейшего расширения функциональности.